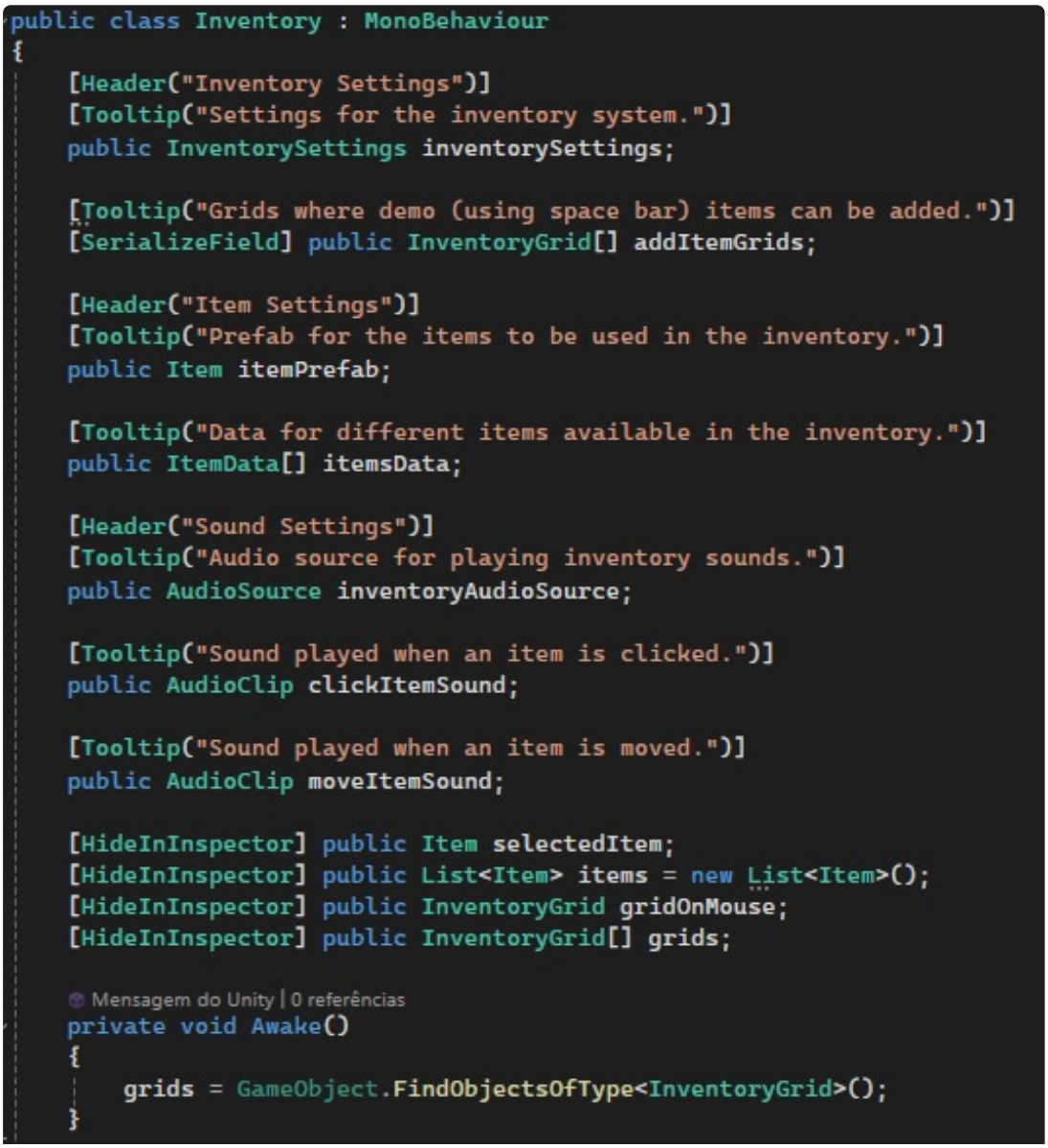
🔌 Components

Here are all the components of the UInventoryGrid project, categorized by methods and functionalities to enhance comprehension.

About

Here are all the scripts included in the project, each with its function. We have provided tooltips and comments within each script to further aid in understanding the system, as shown in the example below.



```
public class Inventory : MonoBehaviour
{
    [Header("Inventory Settings")]
    [Tooltip("Settings for the inventory system.")]
    public InventorySettings inventorySettings;

    [Tooltip("Grids where demo (using space bar) items can be added.")]
    [SerializeField] public InventoryGrid[] addItemGrids;

    [Header("Item Settings")]
    [Tooltip("Prefab for the items to be used in the inventory.")]
    public Item itemPrefab;

    [Tooltip("Data for different items available in the inventory.")]
    public ItemData[] itemsData;

    [Header("Sound Settings")]
    [Tooltip("Audio source for playing inventory sounds.")]
    public AudioSource inventoryAudioSource;

    [Tooltip("Sound played when an item is clicked.")]
    public AudioClip clickItemSound;

    [Tooltip("Sound played when an item is moved.")]
    public AudioClip moveItemSound;

    [HideInInspector] public Item selectedItem;
    [HideInInspector] public List<Item> items = new List<Item>();
    [HideInInspector] public InventoryGrid gridOnMouse;
    [HideInInspector] public InventoryGrid[] grids;

    Mensagem do Unity | 0 referências
    private void Awake()
    {
        grids = GameObject.FindObjectsOfType<InventoryGrid>();
    }
```

```
/// <summary>
/// Selects an item in the inventory.
/// </summary>
/// <param name="item">Item to be selected.</param>
1 referência
public void SelectItem(Item item)
{
    ClearItemReferences(item);
    selectedItem = item;
    selectedItem.rectTransform.SetParent(transform);
    selectedItem.rectTransform.SetAsLastSibling();
}
```

This class provides comprehensive functionality for managing items within an inventory system, including adding, moving, removing, and transferring items.

SizeInt Struct

Represents a 2D size with width and height. Used for defining dimensions within the inventory.

Inventory Class

Manages the inventory system and interactions with items.

Public Variables:

- **Inventory Settings**: Settings for the inventory system.
- **Add Item Grids**: Grids where demo items can be added.
- **Item Prefab**: Prefab for the items to be used in the inventory.
- **Items Data**: Data for different items available in the inventory.
- **Inventory Audio Source**: Audio source for playing inventory sounds.
- **Click Item Sound**: Sound played when an item is clicked.
- **Move Item Sound**: Sound played when an item is moved.

Private Variables:

- **Selected Item**: The currently selected item.
- **Items**: A list of items in the inventory.
- **Grid on Mouse**: The grid where the mouse is currently positioned.
- **Grids**: An array of all inventory grids.

Methods:

- `SelectItem(Item item)` : Selects an item in the inventory.
- `DeselectItem()` : Deselects the currently selected item.
- `AddItem(ItemData itemData, int quantity, bool splitedItem)` : Adds an item to the inventory.
- `AddNewItem(ItemData itemData, int stackCount)` : Creates a new item in the inventory.
- `TryPlaceItemInGrid(InventoryGrid grid, Vector2Int slotPosition, ItemData itemData, int stackCount, bool rotate)` : Attempts to place an item at a specific position within an inventory grid.
- `GetStackableItem(ItemData itemData)` : Retrieves a stackable item that matches the given item data.
- `MoveItem(Item item, bool deselectItemInEnd)` : Moves the specified item within the inventory.
- `ClearItemReferences(Item item)` : Clears item references in the grid.
- `RevertItemReferences(Item item)` : Reverts item references in the grid.
- `ExistsItem(Vector2Int slotPosition, InventoryGrid grid, int width, int height)` : Checks if an item exists at the given slot position.
- `ReachedBoundary(Vector2Int slotPosition, InventoryGrid gridReference, int width, int height)` : Checks if the slot position is within the grid boundaries.
- `IndexToInventoryPosition(Item item)` : Converts an index position to a local position in the inventory.
- `GetSlotAtMouseCoords()` : Gets the slot position at the mouse coordinates.
- `GetItemAtMouseCoords()` : Gets the item at the mouse coordinates.
- `GetItemFromSlotPosition(Vector2Int slotPosition)` : Gets the item at the specified slot position.
- `TransferItemToInventory(Item item, Inventory targetInventory)` : Transfers an item to another inventory.
- `RemoveItem(Item item)` : Removes an item from the inventory.
- `IsItemTypeAllowed(ItemData itemData)` : Checks if the item type is allowed in the current grid.
- `PlayInventoryAudioClip(AudioClip clip)` : Plays an audio clip with random pitch and volume variations.

Inventory Controller

This class is responsible for handling various interactions with an inventory system, including mouse and keyboard inputs, item panel management, item stacking, inspecting items, and using items.

Methods:

- `UpdateGridOnMouse`: Updates the grid currently under the mouse cursor.
- `HandleMouseInput`: Handles mouse input for item interactions such as selecting, moving, and clicking items.
- `HandleKeyboardInput`: Handles keyboard input for inventory actions like adding items, closing panels, and rotating selected items.
- `HandleSelectedItemClick`: Handles click events when an item is selected, including moving, stacking, or transferring items.
- `TransferItemToInventory`: Transfers an item to another inventory grid.
- `HandlePanelMouseClick`: Handles mouse click events on the item panel.
- `UpdateSelectedItemPosition`: Updates the position of the selected item to follow the mouse cursor.
- `TogglePanel`: Toggles the item panel visibility and manages its state.
- `OpenPanel`: Opens the item panel for the specified item.
- `PositionPanel`: Positions the item panel relative to the specified item.
- `SetPanelListeners`: Sets button listeners for the item panel buttons.
- `ClosePanel`: Closes the item panel.
- `ResetCurrentPanelItem`: Resets the current panel item by enabling raycast targets.
- `SetItemsRaycast`: Sets the raycast targets for all items in the inventory.
- `SelectItemWithMouse`: Selects an item at the specified slot position with the mouse.
- `GetItemAtSlot`: Gets the item at the specified slot position.
- `OnItemClick`: Handles item click events.
- `MoveSelectedItemToMouse`: Moves the selected item to follow the mouse cursor.
- `StackItems`: Stacks two items together if possible.

- `RemoveCurrentItem`: Removes the current item with the panel.
- `RemoveItemWithMouse`: Removes the item under the mouse cursor.
- `InspectCurrentItem`: Inspects the current item with the panel.
- `CloseInspectUI`: Closes the inspect UI.
- `SplitItem`: Splits the current item with the panel into two stacks.
- `UseItem`: Uses the current item.
- `CanUseSelectedItem`: Checks if the current item can be used.
- `CloseCurrentPanel`: Closes the current panel.

Inventory Grid

These methods and properties collectively define the behavior and characteristics of the InventoryGrid component within the inventory system.

Inventory Reference:

- `public Inventory inventory`: Holds a reference to the Inventory component, enabling interaction with the main inventory.

Allowed Item Types:

- `public List<ItemType> allowedItemTypes`: List specifying the allowed item types in this grid, allowing item restrictions.

Stored Items:

- `public Item[,] items`: 2D array to store items in the grid, with each cell representing a slot.

Grid Configuration:

- `public Vector2Int gridSize`: Defines the grid size in terms of the number of slots (width, height).
- `public RectTransform rectTransform`: Reference to the RectTransform component for sizing and positioning the grid on the interface.

Initialization:

- `Awake()`: Initializes the grid by creating the items array based on the defined size and setting the RectTransform's size according to the grid size.

Pointer Event Handling:

- `OnPointerEnter(PointerEventData eventData)`: Handles the pointer enter event to set the current grid under the mouse cursor. This allows tracking which grid the mouse is currently over.

Inventory Settings

Overall, this script provides customizable settings for controlling various aspects of the inventory system, such as slot size, animation speed, and slot scale, which can be easily adjusted via the Unity editor.

Slot Settings:

- `public Vector2Int slotSize`: Specifies the size of each inventory slot in pixels, represented as a 2D vector with width and height values.

Item

This script represents an item in the inventory system, providing functionalities for rotation, updating stack count, handling pointer click events, and managing item attributes.

Overview:

The `Item` class encapsulates properties and methods related to individual items within the inventory.

Public Variables:

- **icon**: The icon representing the item.
- **background**: The background image for the item.
- **stackCountText**: The text displaying the stack count.
- **data**: The data of the item.

- **isRotated**: Indicates whether the item is rotated.
- **rotateIndex**: The index representing the current rotation state.
- **stackCount**: The stack count for stackable items.

Properties:

- **indexPosition**: The position of the item in the inventory grid.
- **inventory**: The inventory that contains the item.
- **rectTransform**: The RectTransform component of the item.
- **inventoryGrid**: The inventory grid containing the item.

Methods:

- `Rotate()`: Rotates the item.
- `ResetRotate()`: Resets the item's rotation.
- `UpdateStack()`: Updates the stack count text.
- `OnPointerClick(PointerEventData eventData)`: Handles pointer click events on the item.
- `Start()`: Initializes the item's icon sprite and background color.
- `LateUpdate()`: Updates the rotation animation of the item.
- `UpdateRotateAnimation()`: Updates the rotation animation of the item.
- `UpdateRotation()`: Updates the rotation target based on the rotation index.

Rotation Functionality:

The `Item` class provides methods to rotate items within the inventory grid, allowing for different orientations of items.

Pointer Click Handling:

By implementing the `IPointerClickHandler` interface, the `Item` class handles pointer click events, enabling interaction with items in the inventory.

Stack Count Management:

For stackable items, the `Item` class manages the display and updating of the stack count text, providing visual feedback on the number of stacked items.

Integration with Inventory Controller:

The `Item` class interacts with the `InventoryController` to trigger item click events, facilitating communication between individual items and the inventory system.

ItemData

This script defines the attributes and characteristics of items that can be stored in the inventory, including their name, description, type, size, visual representation, and stackability.

Overview:

The `ItemData` class is a ScriptableObject that stores data for individual items in the inventory system.

Public Variables:

- **itemName**: The name of the item.
- **description**: Description of the item.
- **price**: The price of the item.
- **itemType**: The type/category of the item, defined by the `ItemType` enum.
- **size**: The size of the item in grid units.
- **icon**: The icon representing the item.
- **backgroundColor**: The background color associated with the item.
- **stackable**: Indicates if the item can be stacked.
- **maxStack**: The maximum stack size of the item.

Enum:

- **ItemType**: Enumerates different types/categories of items that can be used to categorize and classify items within the inventory.

CreateAssetMenu Attribute:

- `[CreateAssetMenu(fileName = "ItemData", menuName = "Inventory/ItemData")]`: Specifies that instances of this scriptable object can be created via the Unity editor's CreateAssetMenu. It defines the default file name and the menu path for

creating new instances.

Type Categorization:

Items are categorized into various types such as weapons, ammunition, medical supplies, food and drink, etc., allowing for organization and classification within the inventory system.

Visual Representation:

Each item has an associated icon and background color, providing visual cues to easily identify items in the inventory.

Stackability:

Items can be designated as stackable, allowing multiple instances of the same item to occupy a single inventory slot, up to a maximum stack size defined by the `maxStack` variable.

Draggable Panel UI

This script enables the dragging functionality for UI panels, allowing users to interactively move them within the canvas.

Overview:

The `DraggablePanelUI` class implements the `IPointerDownHandler` and `IDragHandler` interfaces to detect pointer input and handle dragging events on the panel.

Private Variables:

- **panelRectTransform**: Reference to the RectTransform component of the panel.
- **canvas**: Reference to the Canvas component that contains the panel.
- **initialPointerPosition**: The initial position of the pointer when dragging starts.
- **initialPanelPosition**: The initial position of the panel when dragging starts.

Initialization:

In the `Awake` method, references to the RectTransform and Canvas components are obtained to facilitate panel movement.

Pointer Down Handling:

- The `OnPointerDown` method captures the initial position of the pointer when the user clicks on the panel and records the initial position of the panel.

Dragging:

- The `OnDrag` method calculates the offset between the current pointer position and the initial pointer position when dragging starts. It then updates the position of the panel by applying this offset.

End Drag Handling:

- The `OnEndDrag` method can be implemented to perform any actions when dragging of the panel ends, but it's currently empty in the provided script.

Functionality:

- When the user clicks and holds the mouse button on the panel, the `OnPointerDown` method is invoked, capturing the initial pointer position and panel position.
- As the user moves the pointer while holding down the mouse button, the `OnDrag` method continuously updates the position of the panel based on the pointer's movement relative to the initial position.
- When the user releases the mouse button, the dragging of the panel ends, and the `OnEndDrag` method can be used to perform any additional actions if needed (currently empty in the script).

Item Inspect UI

This script represents the user interface (UI) for inspecting items within the inventory system. It provides fields for displaying information about the inspected item, such as its name, description, and price, along with a button to close the inspection UI.

Overview:

The `ItemInspectUI` class manages the visual elements and interaction components of the item inspection UI.

Public Variables:

- **itemName**: Text component for displaying the name of the inspected item.
- **itemDescription**: Text component for displaying the description of the inspected item.
- **itemPrice**: Text component for displaying the price of the inspected item.
- **closeButton**: Button component for closing the item inspection UI.

Functionality:

- **Display Item Information**: The `itemName`, `itemDescription`, and `itemPrice` text components are used to display the corresponding information of the inspected item.
- **Close Button**: The `closeButton` allows users to close the item inspection UI when clicked.

Usage:

- Attach this script to a GameObject that represents the item inspection UI in the scene.
- Assign the appropriate Text and Button components to the public variables in the inspector.
- When an item is inspected, populate the `itemName`, `itemDescription`, and `itemPrice` fields with the relevant information of the inspected item.
- Users can close the item inspection UI by clicking the assigned close button.

Item Panel UI

This script represents the user interface (UI) panel for interacting with selected items within the inventory system. It contains buttons for inspecting, using, splitting, and dropping the selected item, along with a panel to display item inspection details.

Overview:

The `ItemPanelUI` class manages the visual elements and interaction components of the item panel UI.

Public Variables:

- **inspectButton**: Button component for inspecting the selected item.
- **useButton**: Button component for using the selected item.
- **splitButton**: Button component for splitting the stack of the selected item.
- **dropButton**: Button component for dropping the selected item.
- **inspectPanel**: GameObject representing the panel to display item inspection details.

Functionality:

- **Inspect Button**: Allows users to inspect the selected item when clicked.
- **Use Button**: Allows users to use the selected item when clicked.
- **Split Button**: Allows users to split the stack of the selected item when clicked.
- **Drop Button**: Allows users to drop the selected item when clicked.
- **Inspect Panel**: Represents the panel that displays item inspection details when an item is inspected.

Usage:

- Attach this script to a GameObject that represents the item panel UI in the scene.
- Assign the appropriate Button and GameObject components to the public variables in the inspector.
- Implement functionality for each button according to the desired behavior of your inventory system.
- When an item is selected, show the item panel UI and update its visibility based on the available actions for the selected item.

</> Integration

Here we will demonstrate some examples of how to interact with all events of the inventory system.

Add Item

In the `InventoryController` script, we have an example of how to add a random item to the inventory using the space key, which essentially calls the following method:

```
if (Input.GetKeyDown(KeyCode.Space))
{
    inventory.AddItem(inventory.itemsData[Random.Range(0, inventory.itemsDat
}
```

You can add any item to the inventory based on the `ItemData`.

Rotate Item

In the `InventoryController` script, we have an example using the R key to rotate the item, which essentially calls the following method:

```
if (Input.GetKeyDown(KeyCode.R) && inventory.selectedItem != null)
{
    inventory.selectedItem.Rotate();
}
```

Drop Item

To drop an item from the inventory and use this event, we have a function linked to the drop button in the item panel, which essentially calls the following method:

```
itemPanelUI.dropButton.onClick.AddListener(RemoveCurrentItem);
```

And the `RemoveCurrentItem` method:

```
public void RemoveCurrentItem()
{
    if (currentItemWithPanel != null)
    {
        inventory.RemoveItem(currentItemWithPanel);
        itemPanel.SetActive(false);
        currentItemWithPanel = null;
        SetItemsRaycast(true);
    }
}
```

With this function, we get the item that is selected with the mouse click, remove the item, and disable its panel so it doesn't stay open even after the item has been removed. But if you just want to remove the item, you can simply call the method:

```
inventory.RemoveItem(ItemData item);
```

Split and Stack Item

To stack or split items, we use the following functions in `InventoryController`: `SplitItem()` and `StackItems()`. Currently, we have respective buttons in the item panel to call these functions:

```
itemPanelUI.splitButton.onClick.AddListener(SplitItem);
```

And to stack the item, we use a function to control mouse clicks:

```
HandleSelectedItemClick()
```

Within this method, we handle the event of placing one item over another:

```
if (clickedItem != null && clickedItem.data == selectedItem.data && clickedI
{
    StackItems(selectedItem, clickedItem);
}
```

Within this same function, we analyze where the item is being moved, such as to an equipment slot or another container:

```
if (clickedItem != null && clickedItem != selectedItem && clickedItem.invent
{
    if (inventory.IsItemTypeAllowed(selectedItem.data))
    {
        TransferItemToInventory(selectedItem, clickedItem.inventoryGrid);
    }
    else
    {
        Debug.Log("Item type not allowed");
    }
    return;
}
```

Use Item

To use an item, we also have an event linked to the "Use" button in the item panel:

```
itemPanelUI.useButton.onClick.AddListener(UseItem);
```

Which essentially calls the `UseItem()` function. This function validates if the item has the stackable attribute. If so, it removes its quantity from the stack; otherwise, it simply deletes the item.

Another note is that in this demo we validate if the item can be used based on its `ItemType`:

```
private bool CanUseSelectedItem(Item item)
{
    if (item == null)
    {
        return false;
    }

    if(item.data.itemType == ItemType.FoodAndDrink || item.data.itemType ==
    {
        return true;
    }

    return false;
}
```

As you can see, we can consume items of type `FoodAndDrink` and `Medical`. Below is the method that actually uses the item:

```
private void UseItem()
{
    if (currentItemWithPanel != null)
    {
        if (currentItemWithPanel.data.stackable)
        {
            if (currentItemWithPanel.stackCount <= 1)
            {
                Debug.Log("Item cannot be used, it has a stack count of " +
                return;
            }
        }

        if (!CanUseSelectedItem(currentItemWithPanel))
        {
            Debug.Log("Item cannot be used");
            return;
        }

        if (currentItemWithPanel.data.stackable)
        {
            currentItemWithPanel.stackCount--;
            currentItemWithPanel.UpdateStack();

            if(currentItemWithPanel.stackCount <= 0)
            {
                inventory.RemoveItem(currentItemWithPanel);
            }
        }
        else
        {
            inventory.RemoveItem(currentItemWithPanel);
        }
        ClosePanel();
    }
}
```